University of Notre Dame

Department of Computer Science and Engineering

CSE 60745

Graph Learning: Techn & Apps

Assignment Date:	
Submission Date:	
Student Name:	

* This assignment contains 30 points. You are encouraged to finish independently. Any submitted work that is copied from any source or too similar to be an independent write-up will not be given credit. Please submit this homework (as a PDF or Word file including the cover page) to ND Canvas by **11:59pm on Monday, 10/12/2026**.

In this assignment, you will practice the basic usage of PyTorch-Geometric and use the framework to address over-smoothing problem in deep GNNs. Please refer to PyG documentation for more technical details and use Google Colab for experiments. For each question, you will be asked to complete a `MessagePassing()` subclass as shown in the example below, where you need to specify the required components and aggregation strategy in `__init__()`, define propagated message in `message()`, and implement aggregation and node updation in `forward()`. You also need to complete `reset_parameters()` for parameter initialization.

```python
class MyGCNConv(MessagePassing):
    # TODO Q1: implement GCN layer.

    def __init__(self, in_channels, out_channels):
        # in_channels:  dimension of the input node features x, i.e. x.shape[1].
        # out_channels: dimension of the output node representations this layer returns.
        # aggr:         how MessagePassing combines the messages arriving at a node, one of
        #               "add", "mean" or "max". Pick the one that matches the GCN formula.
        super().__init__(aggr=...)
        self.in_channels = in_channels
        self.out_channels = out_channels
        raise NotImplementedError

    def reset_parameters(self):
        # Keep the super() call: it resets the aggregation module, and PyG routes
        # model-wide re-initialisation through here.
        super().reset_parameters()
        raise NotImplementedError

    def forward(self, x, edge_index):
        # Do propagation and updation here.
        raise NotImplementedError

    def message(self, x_j):
        # message` may request any argument you passed to
        # `propagate`, with _j for the source and _i for the target of each edge.
        raise NotImplementedError
```

Q1: GCN Implementation (6 pts)
Complete MyGCNConv(). You can refer to `degree`, `add_self_loops` and `remove_self_loops` functions defined in PyG to assist adjacency matrix related computation.

Q2: GAT Implementation (6 pts)
Complete MyGATConv() in multi-head fashion. Use ` torch_geometric.utils.softmax` for softmax computation.

Q3: GIN Implementation (6 pts)
Complete MyGINConv(). Feel free to design your own MLP architecture.

Q4: R-GCN Implementation (6 pts)
Complete MyRGCNConv(). Please implement the basic version of R-GCN. Edge type labels are provided in `edge_type`.

Q5: Mitigate Over-Smoothing in Deep GNNs (6 pts)
Stacking too many GNN layers will make all node embeddings drift towards each other and cause over-smoothing problem. Please look into the literature on over-smoothing and implement MyDeepConv() to design a GNN model that stays useful with an 8-hop receptive field. Note that 8-hop receptive field is not equal to 8-layer GNN.